Linked List Management System

DSA Mini Project - 2nd Year College Level

Language: Java | Target: Custom Dynamic Node Chaining

1. Executive Summary & Project Overview

The Linked List Management System is an interactive, menu-driven console application for practicing dynamic linear data structures. Implemented from scratch in Java, it contains a full-featured Singly Linked List (19 operations) and Doubly Linked List (20 operations), adhering to all academic specifications.

2. Language Choice Justification: Java

- Object Reference Model: Uses explicit object chaining (node.next, node.prev) avoiding raw pointer arithmetic pitfalls while retaining dynamic memory modeling.
- Automatic Garbage Collection: Safely reclaims memory for unlinked nodes via JVM GC root reachability, preventing memory leaks and dangling references.
- Strong Typing & Robust I/O: Encapsulates static Node classes with Scanner validation.

3. System Architecture & Components

The system is structured into modular components:

1. SinglyLinkedList.java: Unidirectional list handling 19 core operations.
2. DoublyLinkedList.java: Bidirectional list maintaining head and tail references.
3. Main.java: Interactive console menu with full input validation & comparison UI.
4. TestRunner.java: Automated regression harness validating all operations & edge cases.

4. Operations Summary: Singly Linked List

1. Insert Beginning O(1) | 8. Delete Position O(n) | 15. Find Minimum O(n)
2. Insert End O(n) | 9. Delete Value O(n) | 16. Middle Element O(n)
3. Insert Position O(n) | 10. Search Value O(n) | 17. Reverse List O(n)
4. Insert After Node O(n) | 11. Update Value O(n) | 18. Bubble Sort O(n^2)
5. Insert Before NodeO(n) | 12. Display List O(n) | 19. Remove Dups O(n^2)
6. Delete Beginning O(1) | 13. Count Nodes O(n) | 20. Exit
7. Delete End O(n) | 14. Find Maximum O(n)

5. Operations Summary: Doubly Linked List

Supports all operations above with O(1) End Insertion and End Deletion using tail, plus Op 12: Display Forward and Op 13: Display Backward (NULL <- 40 <-> 30 -> NULL).

Comparative Analysis & Technical Evaluation

6. Singly vs Doubly Linked List Comparison

Feature / Dimension	Singly Linked List (SLL)	Doubly Linked List (DLL)
Node Structure	data, next	prev, data, next
Pointer Overhead	1 reference per node	2 references per node
Traversal Direction	Forward only (Unidirectional)	Forward and Backward
Insert at End	O(n) without tail	O(1) with tail reference
Delete from End	O(n) (requires predecessor)	O(1) with tail reference
Delete given node ref	O(n)	O(1) (node.prev.next = node.next)
Reverse Complexity	O(n) time, O(1) space	O(n) time, O(1) space
Typical Applications	Stacks, Queues, Hash Chaining	Browser History, Undo/Redo, LRU

7. Key Algorithm Highlights

- Middle Element: Floyd's slow/fast pointers. For even-length lists (2k nodes), rule selects the upper/second middle (pos k+1) while reporting both central nodes.
- In-Place Reversal: Re-links next in SLL; swaps prev and next in DLL in O(1) space.
- Bubble Sort: In-place data swapping with early-termination swapped flag optimization.
- Duplicate Removal: Runner pointer O(n^2) time, O(1) space, preserves valid links.

8. Edge Cases Tested (Section 10 Brief)

1. Empty list operations (delete, search, update, min/max, middle, reverse, sort).
2. Single-node list state consistency (head == tail, reversal/sort identity).
3. Insert/delete at position 1 (head updates).
4. Insert at last valid position (count + 1) updating tail.
5. Invalid positions (0, out of bounds) intercepted with error diagnostics.
6. Non-existent value deletion & search reporting not found.
7. Multiple and consecutive duplicates completely sanitized.
8. Negative numbers and zero correctly evaluated in min, max, and sorting.
9. Sorting empty or 1-node lists safely returns without error.
10. Reversing empty or 1-node lists safely returns without error.

9. Test Verification & Conclusion

All test suites run via 'make test' or './test.sh'. Full output logged in sample-output/output.txt. The project achieves 100% adherence to all project briefs.